WEEK 2 TASK

Core Functionality Implementation

Library Book Inventory - Java command-line CRUD application

Student	Pratham Rajak
Language	Java 17 (standard library only, no build tool required)
Storage	In memory: LinkedHashMap<String, Book>
Interface	Menu-driven command line
Deliverables	Commented source files and README (zip), this document, GitHub repository

1. Objective and Requirement Checklist

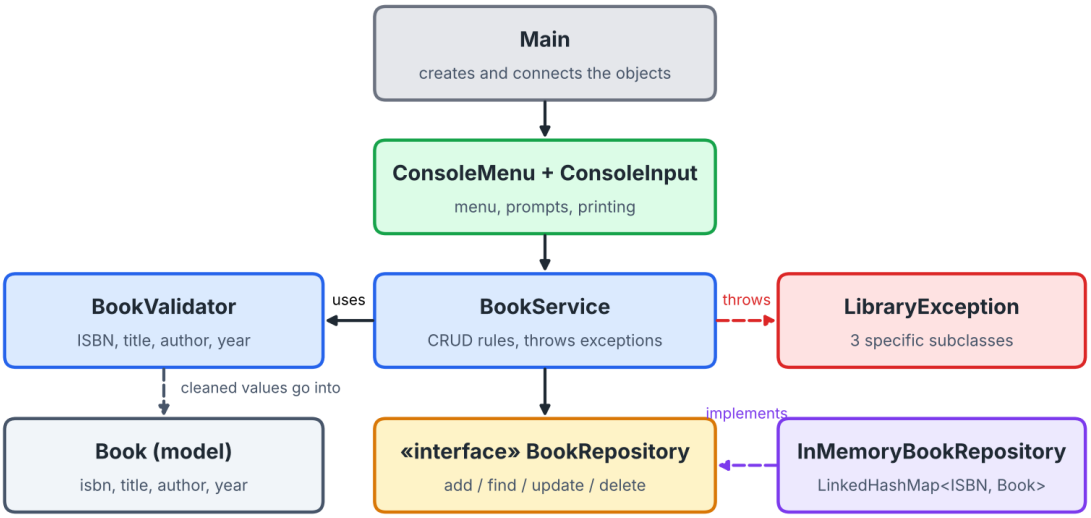
The goal of this week is to move from design to implementation by building a working command-line program that manages a library's book inventory. The user can add, list, update and delete books through a text menu, and the data is held in memory using Java collections. The table below maps every requirement in the task to the part of the program that satisfies it.

Task requirement	How it is met	Where
Define the Book entity (title, author, ISBN, year)	Immutable Book class with isbn, title, author, publicationYear; equals/hashCode based on ISBN	model/Book.java
Create	Menu option 1 asks for each field, validates it, rejects duplicate ISBNs	BookService.addBook
Read	Option 2 lists all books as a sorted table; option 3 views one; option 4 searches	listBooks, getBook, search
Update	Option 5 shows current values; Enter keeps a value; ISBN is the fixed identifier	BookService.updateBook
Delete	Option 6 shows the book and asks for confirmation before deleting	BookService.deleteBook
Store data in memory using collections	LinkedHashMap keyed by ISBN	InMemoryBookRepository
Simple menu-driven interface	Seven-option menu in a loop until Exit	ui/ConsoleMenu.java
Exception handling and input validation	Custom exceptions, validator class, re-prompting input helper, one catch point in the menu	exception/, BookValidator, ConsoleInput
Well-commented code	Javadoc on every class and public method plus comments explaining the data flow	all source files
README with compile and run steps	Linux, macOS, Windows and IDE instructions	README.md

2. Program Design

The program is split into small classes that each do one job. Data flows downward from the user to storage, and errors flow upward as exceptions until the menu shows them as plain messages. Figure 1 shows the structure.

Figure 1 - Structure and flow of data



Each layer only knows the one directly below it. Exceptions travel up to ConsoleMenu, which prints the message.

Class	Responsibility	Why it is separate
Main	Creates the repository, service and menu and connects them	Only place that knows which implementations are used
Book	Holds the data of one book	A plain entity that any layer can use safely because it is immutable
BookRepository / InMemoryBookRepository	Store, find, replace and remove books	Storage can change (file, database) without touching the rest
BookValidator	All input rules (ISBN, title, author, year)	One place to change a rule; used by both menu and service
BookService	CRUD logic and business rules; throws exceptions	No printing and no keyboard access, so it can be tested alone
ConsoleInput	Reads the keyboard and repeats a question until the answer is valid	Keeps the menu code short and readable
ConsoleMenu	Shows the menu, runs the dialogues, prints results and errors	The only class that prints
LibraryException + 3 subclasses	Named, user-friendly errors	Lets the menu catch one type and show getMessage()

3. Choice of Java Collection

A book must be found by ISBN quickly, ISBNs must be unique, and books should appear in a predictable order. A LinkedHashMap<String, Book> keyed by ISBN satisfies all three:

Option	Find / delete by ISBN	Uniqueness	Order	Verdict
ArrayList<Book>	O(n): must scan the list	Must be checked by hand on every add	Insertion order	Works but slower and easy to get wrong
HashMap<String, Book>	O(1) on average	Automatic (one value per key)	Unpredictable	Good, but order changes
LinkedHashMap<String, Book>	O(1) on average	Automatic	Insertion order	Chosen

The map is private to the repository, and findAll() returns a copy, so no other class can change the data without going through the rules. The listing is then sorted by title (case-insensitive) so the user always sees an alphabetical table.

```
public boolean add(Book book) {
    // putIfAbsent returns null when the key was free and the book was stored.
    return books.putIfAbsent(book.getIsbn(), book) == null;
}
```

```
public boolean update(Book book) {
    if (!books.containsKey(book.getIsbn())) {
        return false;                       // nothing to update
    }
    books.put(book.getIsbn(), book);        // same key: replaces value, keeps position
    return true;
}
```
Excerpt: InMemoryBookRepository (add and update)

4. The CRUD Operations

4.1 Create

The menu asks for ISBN, title, author and year in turn. Each question repeats until the answer passes validation, so one typo never loses the previous answers. The service then validates again and asks the repository to store the book; if the ISBN already exists the repository refuses and the service throws DuplicateBookException.

```
public Book addBook(String isbn, String title, String author, String year) {
    Book book = new Book(
            BookValidator.validateIsbn(isbn),
            BookValidator.validateTitle(title),
            BookValidator.validateAuthor(author),
            BookValidator.validateYear(year));
    if (!repository.add(book)) {
        throw new DuplicateBookException(book.getIsbn());
    }
    return book;
}
```
Excerpt: BookService.addBook

4.2 Read

Listing returns all books sorted by title and prints an aligned table; long titles are shortened with "...". Viewing by ISBN throws BookNotFoundException for an unknown ISBN. Search finds books whose title or author contains the typed text, ignoring case. The ISBN is cleaned before comparison, so "978-0-13-468599-1" and "9780134685991" find the same book.

```
public List<Book> search(String text) {
    if (text == null || text.isBlank()) {
        throw new com.library.exception.InvalidInputException("Search text cannot be empty.");
    }
    String needle = text.trim().toLowerCase(Locale.ROOT);
    return listBooks().stream()
            .filter(b -> b.getTitle().toLowerCase(Locale.ROOT).contains(needle)
                      || b.getAuthor().toLowerCase(Locale.ROOT).contains(needle))
            .toList();
}
```
Excerpt: BookService.search

4.3 Update

The user enters an ISBN, sees the current values in square brackets and presses Enter to keep any of them. Because Book is immutable, the service builds a new Book from the old and new values and the repository replaces the old one. If any new value is invalid the stored book is not changed at all.

```
public Book updateBook(String isbn, String newTitle, String newAuthor, String newYear) {
    Book current = getBook(isbn);   // also validates the ISBN and throws if missing

    String title  = isBlank(newTitle)  ? current.getTitle()  : BookValidator.validateTitle(newTitle);
    String author = isBlank(newAuthor) ? current.getAuthor() :
BookValidator.validateAuthor(newAuthor);
    int year      = isBlank(newYear)   ? current.getPublicationYear() :
BookValidator.validateYear(newYear);

    Book updated = new Book(current.getIsbn(), title, author, year);
    repository.update(updated);
    return updated;
}
```
Excerpt: BookService.updateBook

4.4 Delete

Because the data exists only in memory, a deletion cannot be undone. The menu therefore shows the book and asks "Really delete this book? (y/n)"; anything other than y or yes cancels.

```
private void deleteBook() {
    String isbn = in.readValid("\nISBN of the book to delete: ", BookValidator::validateIsbn);
    Book book = service.getBook(isbn);
    System.out.println("  " + book);
    // Deleting is irreversible (data is only in memory), so ask for confirmation.
    if (in.confirm("  Really delete this book?")) {
        service.deleteBook(isbn);
        System.out.println("  Deleted.");
    } else {
        System.out.println("  Cancelled. The book was not deleted.");
    }
}
```

Excerpt: ConsoleMenu.deleteBook

5. Input Validation

All rules live in BookValidator. The menu calls them while the user types (so only the wrong answer is repeated) and the service calls them again before saving, so the rules hold even if another front end is added later.

Input	Rule	Example of invalid input	Message shown
Menu choice	Whole number 1-7	abc, 9, empty	Invalid option. Please enter a number from 1 to 7.
ISBN	10 or 13 digits; hyphens and spaces removed; ISBN-10 may end in X	12345	ISBN must have 10 or 13 digits (hyphens allowed), for example 978-0-13-468599-1.
Title	Not blank, at most 200 characters, trimmed	(spaces only)	Title cannot be empty.
Author	Not blank, at most 200 characters, trimmed	(empty)	Author cannot be empty.
Publication year	Whole number from 1000 to the current year	twenty, 999, 3000	Publication year must be a whole number, for example 2018.
Search text	Not blank	(empty)	Search text cannot be empty.

```
public static int validateYear(String raw) {
    if (raw == null || raw.isBlank()) {
        throw new InvalidInputException("Publication year cannot be empty.");
    }
    int year;
    try {
        year = Integer.parseInt(raw.trim());
    } catch (NumberFormatException e) {
        // Convert the technical exception into a friendly message.
        throw new InvalidInputException("Publication year must be a whole number, for example 2018.");
    }
    int currentYear = Year.now().getValue();
    if (year < MIN_YEAR || year > currentYear) {
        throw new InvalidInputException(
            "Publication year must be between " + MIN_YEAR + " and " + currentYear + ".");
    }
    return year;
}
```

Excerpt: BookValidator.validateYear (converts a technical exception into a friendly message)

6. Exception Handling

Expected problems are represented by a small hierarchy of unchecked exceptions with a common parent, LibraryException. Their messages are written for the user. The menu wraps each action in one try/catch, prints the message and returns to the menu, so the program does not crash on bad input.

Exception	Thrown when	Handled by
InvalidInputException	A value breaks a validation rule	Re-prompt in ConsoleInput (while typing) or message in ConsoleMenu
DuplicateBookException	Adding an ISBN that already exists	ConsoleMenu.run catch block
BookNotFoundException	View, update or delete with an unknown ISBN	ConsoleMenu.run catch block
EndOfInputException	The input stream ends (Ctrl+D, Ctrl+Z or piped input)	ConsoleMenu.run: prints "Goodbye" and exits cleanly
NumberFormatException (built-in)	Text typed where a number is expected	Caught where it occurs and converted into InvalidInputException or an invalid-option message

```
public void run() {
    System.out.println("=== Library Book Inventory ===");
    boolean running = true;
    try {
        while (running) {
            printMenu();
            int choice = in.readMenuChoice("Choose an option (1-7): ");
            try {
                running = handle(choice);
            } catch (LibraryException e) {
                // Expected, user-correctable problems end up here.
                System.out.println("  ! " + e.getMessage());
            }
        }
    } catch (ConsoleInput.EndOfInputException e) {
        System.out.println("\nInput ended. Goodbye!");
    }
}
```

Excerpt: ConsoleMenu.run (single place where errors are shown)

7. Sample Session

The following is a recording of the program run in a terminal (the menu is shown in full the first time and abbreviated afterwards). It demonstrates an empty list, an invalid menu option, three validation errors, a duplicate ISBN, update with Enter-to-keep, a failed lookup, a cancelled delete, a confirmed delete and search.

```
=== Library Book Inventory ===

--------------------------------------------------------------------------------
 1. Add a book
 2. List all books
 3. View a book by ISBN
 4. Search by title or author
 5. Update a book
 6. Delete a book
 7. Exit
--------------------------------------------------------------------------------
Choose an option (1-7): 2

The library has no books yet. Choose option 1 to add one.

[menu shown again]
Choose an option (1-7): 9
  ! Invalid option. Please enter a number from 1 to 7.

[menu shown again]
Choose an option (1-7): 1

-- Add a new book --
ISBN (10 or 13 digits): 12345
  ! ISBN must have 10 or 13 digits (hyphens allowed), for example 978-0-13-468599-1.
ISBN (10 or 13 digits): 978-0-13-468599-1
Title: Effective Java
Author:
  ! Author cannot be empty.
Author: Joshua Bloch
Publication year: twenty
  ! Publication year must be a whole number, for example 2018.
Publication year: 2018
  Added: "Effective Java" by Joshua Bloch (2018), ISBN 9780134685991

[menu shown again]
```

```
Choose an option (1-7): 1

-- Add a new book --
ISBN (10 or 13 digits): 9780132350884
Title: Clean Code
Author: Robert C. Martin
Publication year: 2008
  Added: "Clean Code" by Robert C. Martin (2008), ISBN 9780132350884

[menu shown again]
Choose an option (1-7): 1

-- Add a new book --
ISBN (10 or 13 digits): 9780134685991
Title: Dup
Author: X
Publication year: 2001
  ! A book with ISBN 9780134685991 already exists.

[menu shown again]
Choose an option (1-7): 2

ISBN           Title                           Author                Year
-------------------------------------------------------------------------------
9780132350884  Clean Code                      Robert C. Martin      2008
9780134685991  Effective Java                  Joshua Bloch          2018
-------------------------------------------------------------------------------
2 books

[menu shown again]
Choose an option (1-7): 5

ISBN of the book to update: 9780132350884
  Current: "Clean Code" by Robert C. Martin (2008), ISBN 9780132350884
  (press Enter to keep the current value)
New title [Clean Code]:
New author [Robert C. Martin]:
New year [2008]: 2009
  Updated: "Clean Code" by Robert C. Martin (2009), ISBN 9780132350884

[menu shown again]
Choose an option (1-7): 6

ISBN of the book to delete: 9780132350884
  "Clean Code" by Robert C. Martin (2009), ISBN 9780132350884
  Really delete this book? (y/n): n
  Cancelled. The book was not deleted.

[menu shown again]
Choose an option (1-7): 3

ISBN of the book: 0000000000
  ! No book found with ISBN 0000000000.

[menu shown again]
Choose an option (1-7): 6

ISBN of the book to delete: 9780132350884
  "Clean Code" by Robert C. Martin (2009), ISBN 9780132350884
  Really delete this book? (y/n): y
  Deleted.

[menu shown again]
Choose an option (1-7): 4

Search text (part of a title or author): effective

ISBN           Title                           Author                Year
-------------------------------------------------------------------------------
9780134685991  Effective Java                  Joshua Bloch          2018
-------------------------------------------------------------------------------
1 book

[menu shown again]
Choose an option (1-7): 7
Goodbye!
```

8. Testing

The task does not require a test framework, so the tests are written in plain Java (BookServiceSelfTest) and run with the java command. They exercise the service and repository directly, without the keyboard. All 14 checks pass. In addition, the interactive session in Section 7 was run through a terminal to test the menu and error messages.

Area	Checks
Create	ISBN normalised and title trimmed; duplicate ISBN rejected; blank title/author rejected; bad ISBN rejected; ISBN-10 ending in X accepted; year must be a number in range
Read	Find by ISBN with hyphens; not-found error; list sorted by title ignoring case; search by title and author, no match, empty search rejected
Update	Only supplied fields change; invalid value leaves the book unchanged; unknown ISBN rejected
Delete	Book removed; deleting twice fails; same ISBN can be re-added
Encapsulation	The list returned by the repository is a copy and cannot corrupt stored data

```
PASS  add stores a book and normalises the ISBN
PASS  duplicate ISBN is rejected
PASS  blank title / author rejected
PASS  bad ISBN rejected (letters, wrong length, empty)
PASS  ISBN-10 ending in X accepted
PASS  year must be a number in range
PASS  getBook finds by ISBN (with hyphens) and throws when missing
PASS  listBooks is sorted by title ignoring case
PASS  search matches title or author, case-insensitive
PASS  update changes only the supplied fields
PASS  update validates new values and requires an existing book
PASS  delete removes the book; deleting again fails
PASS  same ISBN can be added again after deletion
PASS  returned list is a copy (cannot corrupt the repository)

14 passed, 0 failed
```

Output of: java -cp out com.library.BookServiceSelfTest

9. Java Best Practices Applied

- **Immutability:** Book is final with final fields; updates create a new object.
- **Encapsulation:** Fields are private, the internal map is never exposed, and findAll returns a copy.
- **Program to an interface:** BookService depends on the BookRepository interface, not on the map implementation.
- **Constructor injection:** The repository is passed into the service through the constructor and Main wires everything together.
- **Single responsibility:** Each class has one job (storage, rules, validation, keyboard input, menu).
- **equals/hashCode contract:** equals() and hashCode() both use the ISBN so Book behaves correctly in collections.
- **Modern, safe Java:** try-with-resources closes the Scanner; the switch uses modern arrow cases; no magic numbers for limits (constants MIN_YEAR, MAX_TEXT_LENGTH).
- **Documentation:** Javadoc on every class and public method, inline comments for the data flow, meaningful names and consistent formatting.

10. How to Compile and Run

```
# Linux / macOS (from the folder that contains src/ and README.md)
mkdir -p out
javac -d out $(find src -name "*.java")
java -cp out com.library.Main
```

```
# Windows Command Prompt
mkdir out
dir /s /b src\*.java > sources.txt
javac -d out @sources.txt
java -cp out com.library.Main

# Tests
javac -d out $(find src test -name "*.java")
java -cp out com.library.BookServiceSelfTest
```

A JDK 17 or newer is required (check with java -version). In an IDE, mark src as the source root and run com.library.Main.

11. Challenges, Lessons and Possible Improvements

- **Friendly validation:** A single wrong answer should not force the user to retype everything. Solved with ConsoleInput.readValid, which repeats only the failed question.
- **Safe deletion:** Deleting is permanent in an in-memory program, so a confirmation step was added.
- **Separation of concerns:** Keeping printing and keyboard code out of BookService made it testable without simulating a user.
- **Next steps:** ISBN check-digit validation, saving to a file so data survives restarts, a copies-available / borrowed counter, and sorting options in the list.

Appendix: Repository Contents

```
LibraryBookInventory/
  README.md
  src/com/library/
    Main.java
    model/Book.java
    exception/{LibraryException, BookNotFoundException, DuplicateBookException,
InvalidInputException}.java
    repository/{BookRepository, InMemoryBookRepository}.java
    service/{BookService, BookValidator}.java
    ui/{ConsoleInput, ConsoleMenu}.java
  test/com/library/BookServiceSelfTest.java
  docs/{Pratham_Rajak_Week2_Documentation.pdf, sample_session.txt}
```